

Decentralized Computing and Blockchains

L2 scalability

Alysson Bessani
DI-FCUL

With examples from Dan Boneh's (Stanford) slides:
<https://cs251.stanford.edu/lectures/lecture17.pdf>
<https://cs251.stanford.edu/lectures/lecture18.pdf>

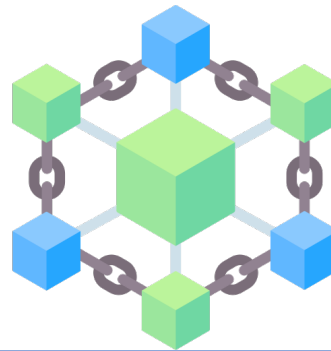
Motivation

- Low performance and high transaction fee of popular blockchains
 - Bitcoin processes 7 tx/s with an inter-block time of 10 min
 - Ethereum processes less than 41 tx/s with an inter-block time of 12 sec
 - 178 tx/s is the theoretical maximum, given the maximum of 45M gas per block, and the typical cost of a simple tx being 21k gas.
 - Transaction fees goes from \$1 to \$2 in Bitcoin and \$0.04 to \$0.20 in Ethereum
- Non-blockchain payment services:
 - VISA: 24k tx/s (2k regularly)
 - Paypal: 200 tx/s
- How can we make blockchains faster and cheaper without changing their core protocols?

Blockchain Layers

The core idea is to avoid using expensive L1 blockchains for each transaction, but still rely on the security of the L1 blockchain

Layer 1

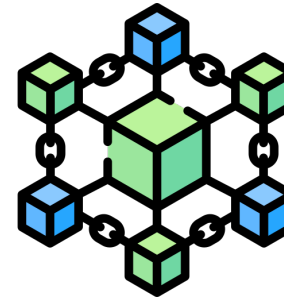


Typical
blockchains

Layer 2

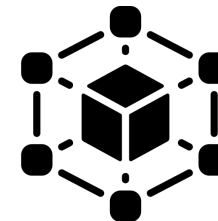


Payment
networks



Rollups
L2 blockchains

Layer 3

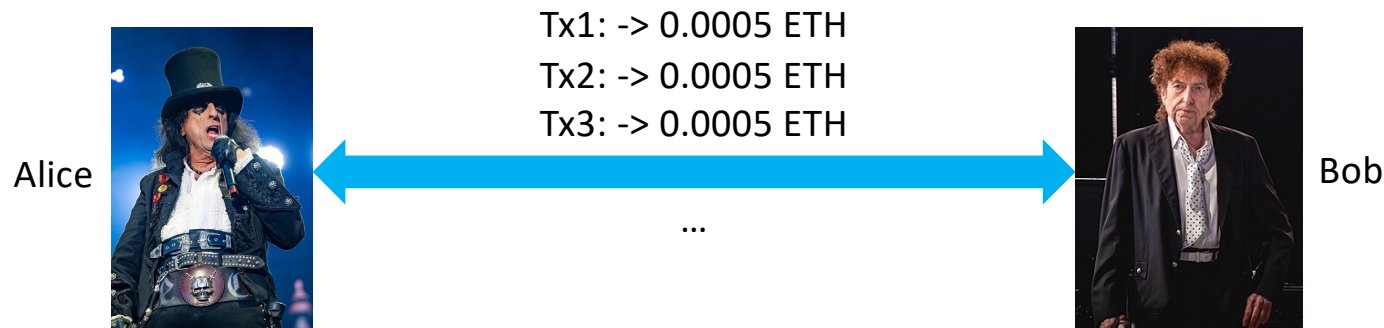


Highly customized
blockchains

Payment Channels

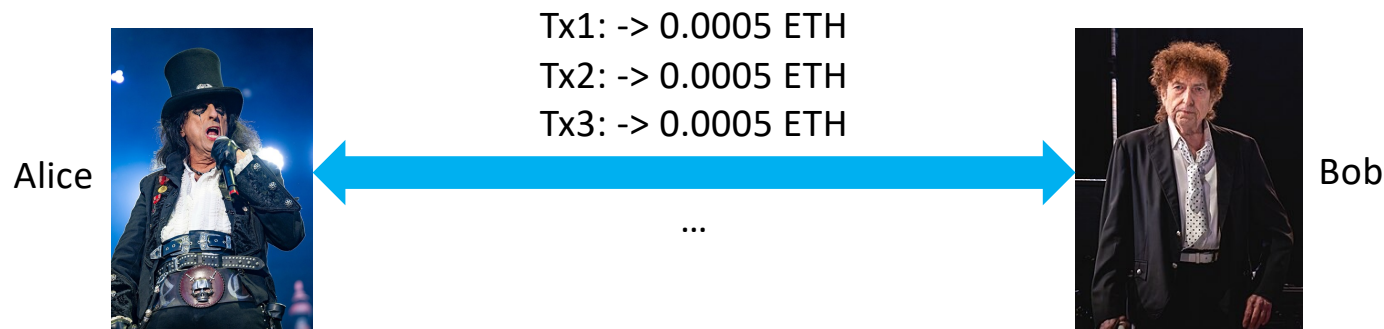
General Idea

- Assume Alice regularly buy coffee from Bob:



- For each tx, they must wait for the blockchain and pay the fee
- What if, instead, Alice deposits 0.05ETH to Bob and, they exchange txs only between themselves.
- At the end of the month, Bob refunds the unused deposit to Alice.
 - Just two blockchain txs per month!

General Idea: complications



- What exactly they exchange?
 - A state update on the channel (A: 0.05, B:0), (A:0.0495, B:0.0005), ...
- How to ensure both parties will follow the protocol? E.g., how to ensure Bob will return the remaining ETH?
 - Create a failsafe mechanism, a **timelock**: if both sign the final balance, the funds are split, otherwise, Alice receives everything if the channel expires

Payment Channels on Ethereum



```
3 contract SimplePaymentChannel {
4     address payable public sender;    // The account sending payments.
5     address payable public recipient; // The account receiving the payments.
6     uint256 public expiration; // Timeout in case the recipient never closes.
7
8     constructor (address payable _recipient, uint256 duration)
9         public
10        payable
11    {
12        sender = msg.sender;
13        recipient = _recipient;
14        expiration = now + duration;
15    }
16
17    /// the recipient can close the channel at any time by presenting a
18    /// signed amount from the sender. the recipient will be sent that amount,
19    /// and the remainder will go back to the sender
20
21    function close(uint256 amount, bytes memory signature) public {
22        require(msg.sender == recipient);
23        require(isValidSignature(amount, signature));
24
25        recipient.transfer(amount);
26        selfdestruct(sender);
27    }
28
29    /// if the timeout is reached without the recipient closing the channel,
30    /// then the Ether is released back to the sender.
31    function claimTimeout() public {
32        require(now >= expiration);
33        selfdestruct(sender);
34    }
35 }
```

The channel is a contract!
(this example uses deprecated calls)

← Alice creates contract with funds,
specifies timelock and recipient

← verify Alice's signature on
final amount.
Only Bob can call close() !!

← send all funds to sender after timelock

Bidirectional payment channels in

- A bi-directional channel is better than two unidirectional channel, as unidirectional channels exhaust faster
- Updates on the channel are signed by the sender and sent to the recipient
- In the end, the channel is closed with (hopefully) the most recent state
- Example: consider a 1ETH channel:
 - Initial state: (A: 0.5, B: 0.5, Nonce=0) on the smart contract
 - B ->0.1-> A: (A: 0.6, B:0.4, Nonce=1) signed by B, sent to A
 - A ->0.2-> B: (A: 0.4, B:0.6, Nonce=2) signed by A, sent to B
 - A ->0.1-> B: (A: 0.3, B:0.7, Nonce=3) signed by A, sent to B
 - ...
 - Eventually Alice (resp. Bob) wants to close the channel:
 - She sends the latest balance to the contract and starts a challenge period
 - If Bob does nothing, Alice's final state is used to distribute funds
 - If Bob submits a state with higher nonce, this is the used state

Summary: Bidirectional channels in

- Updatable smart contracts allows the simple implementation of channels:
 - One/Two on-chain tx for opening the channel (deposit funds)
 - Alice and Bob exchange funds by sending signed state updates to each other
 - One/two on chain tx for closing the channel (distribute funds)
- Payment channels can be generalized to state channels
 - Begin and end channel: on chain
 - All moves done off-chain
 - State channels can be used to implement any 2-party contract off chain

Payment Channels on Bitcoin

- Bidirectional channels are easily implemented in Ethereum due to the updatable account abstraction it supports
 - So, the same idea can be used in other blockchains with updatable accounts
- In Bitcoin, the implementation is more complicated as there are no accounts, just UTXOs
- Recall the UTXO model: there are input and output objects, which are consumed and generated when a *tx* is processed
- Since state cannot be updated, the solution is to get a *tx* that invalidates the previous state

(From a previous lecture)

Preface: Bitcoin Programming

- Bitcoin supports a simple scripting language to program transactions
- This language, called Script, assumes a stack-based machine (see <https://en.bitcoin.it/wiki/Script>)
 - The language is quite limited: no loops, simple operations for validating transactions

```
scriptPubKey: <expiry time> OP_CHECKLOCKTIMEVERIFY OP_DROP OP_DUP OP_HASH160  
<pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG
```

```
scriptSig: <sig> <pubKey>
```

Stack	Script	Description
Empty.	<sig> <pubKey> OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG	scriptSig and scriptPubKey are combined.
<sig> <pubKey>	OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG	Constants are added to the stack.
<sig> <pubKey> <pubKey>	OP_HASH160 <pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG	Top stack item is duplicated.
<sig> <pubKey> <pubHashA>	<pubKeyHash> OP_EQUALVERIFY OP_CHECKSIG	Top stack item is hashed.
<sig> <pubKey> <pubHashA> <pubKeyHash>	OP_EQUALVERIFY OP_CHECKSIG	Constant added.
<sig> <pubKey>	OP_CHECKSIG	Equality is checked between the top two stack items.
true	Empty.	Signature is checked for top two stack items.

UTXO Payment Channels

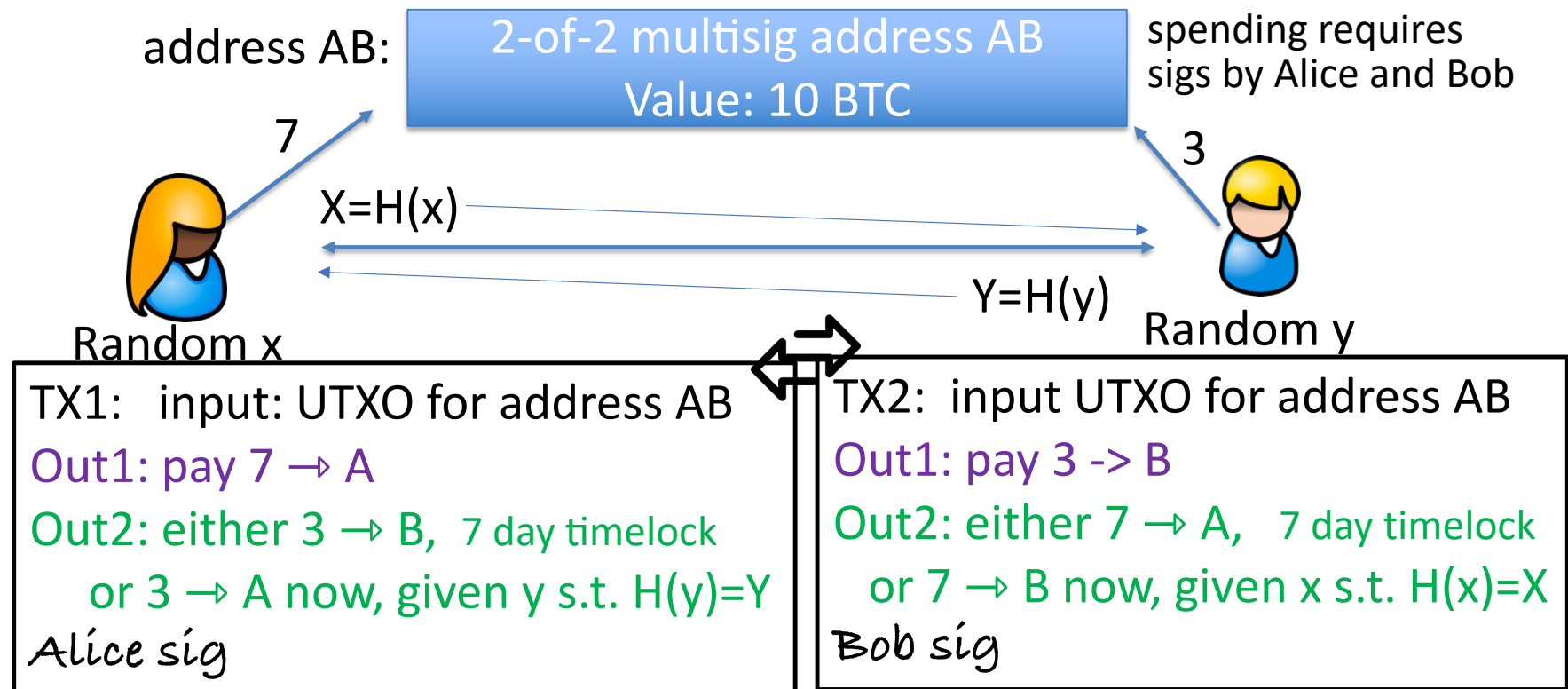
- To implement a channel, create an UTXO with two information (t, X) that can be spent in one of two ways:
 - **Relative time-lock**: a properly signed tx can spend this UTXO $\geq t$ blocks after it was created
 - **Hash lock**: a properly signed tx can spend this UTXO immediately by presenting x , s.t., $X = \text{SHA256}(x)$
- This is called a **Hash-TimeLock Contract (HTLC)**

```
OP_IF      // Alice can redeem UTXO any time using a preimage
  OP_HASH256 <digest> OP_EQUALVERIFY          // redeem by providing <digest> preimage,
  DUP HASH256 <AlicePKhash> EQVERIFY CHECKSIG // and Alice's signature

OP_ELSE    // Bob can redeem UTXO only after timelock
  <num> OP_CLTV OP_DROP                      // redeem <num> blocks after UTXO created,
  DUP HASH256 <BobPKhash> EQVERIFY CHECKSIG  // and Bob's signature

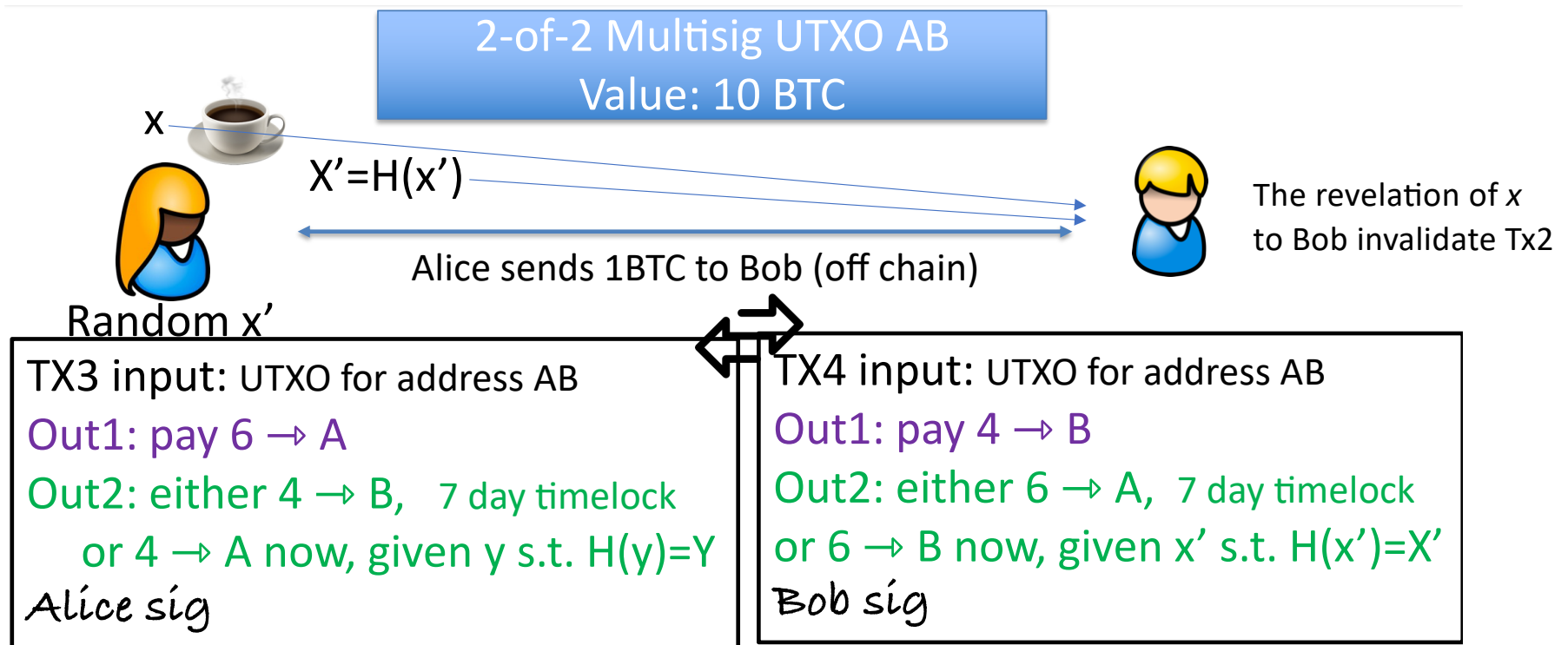
OP_ENDIF
```

UTXO Payment Channel: setup



Alice can sign and post Tx2, wait 7 days, and get her funds back. Same for Bob.

UTXO Payment Channel: transferring funds



UTXO Payment Channel: closing

- Good case:
 - Alice (resp. Bob) posts Tx4 (resp. Tx3) to chain and close the channel after 7 days
 - Alice gets 6
 - Bob gets 4

Alice has TX2, TX4, x , x'

TX2: (stale state)
pay 3 $\rightarrow$ B
either 7 $\rightarrow$ A, 7 day timelock
or 7 $\rightarrow$ B now, given x s.t. $H(x)=X$
Bob sig

TX4: (current state)
pay 4 $\rightarrow$ B
either 6 $\rightarrow$ A, 7 day timelock
or 6 $\rightarrow$ B now, given x' s.t. $H(x')=X'$
Bob sig

Bob has TX1, TX3, y , x

TX1: (stale state)
pay 7 $\rightarrow$ A
either 3 $\rightarrow$ B, 7 day timelock
or 3 $\rightarrow$ A now, given y s.t. $H(y)=Y$
Alice sig

TX3: (current state)
pay 6 $\rightarrow$ A
either 4 $\rightarrow$ B, 7 day timelock
or 4 $\rightarrow$ A now, given y s.t. $H(y)=Y$
Alice sig

UTXO Payment Channel: closing

- **Bad case:**
 - If Alice posts Tx2, Bob uses x
 - Alice gets 0
 - Bob gets 10
- *In general, each time a party spends, it needs to invalidate its previous Tx by revealing the preimage of the hash*

Alice has TX2, TX4, x, x'

TX2: (stale state)
 pay 3 → B
 either 7 → A, 7 day timelock
 or 7 → B now, given x s.t. $H(x)=X$
 Bob sig

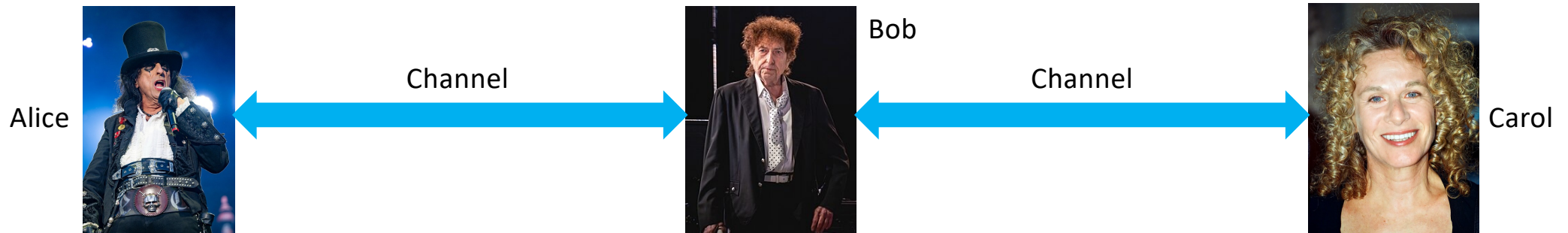
TX4: (current state)
 pay 4 → B
 either 6 → A, 7 day timelock
 or 6 → B now, given x' s.t. $H(x')=X'$
 Bob sig

Bob has TX1, TX3, y, x

TX1: (stale state)
 pay 7 → A
 either 3 → B, 7 day timelock
 or 3 → A now, given y s.t. $H(y)=Y$
 Alice sig

TX3: (current state)
 pay 6 → A
 either 4 → B, 7 day timelock
 or 4 → A now, given y s.t. $H(y)=Y$
 Alice sig

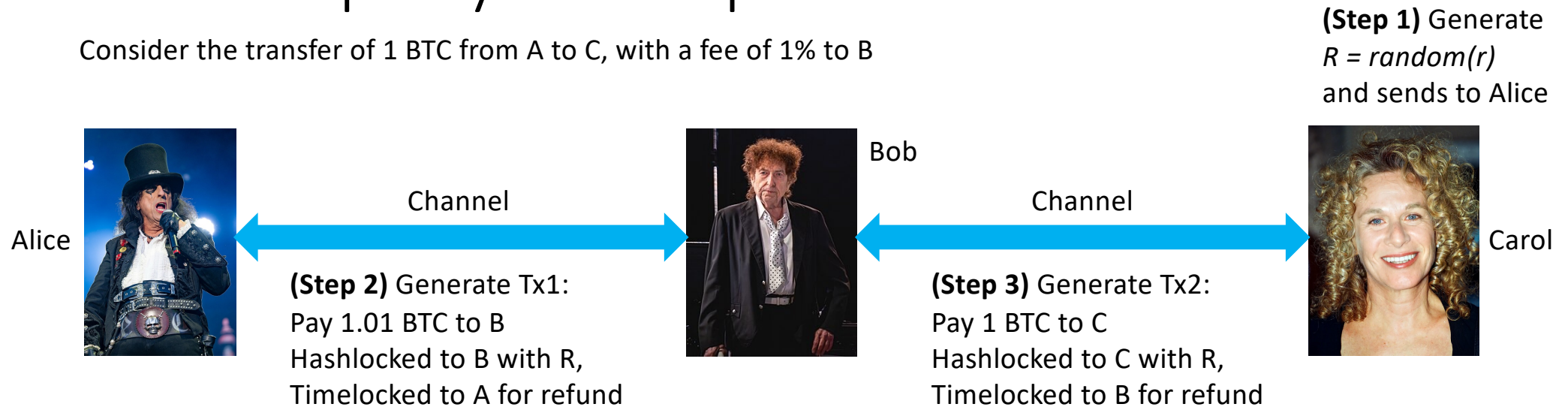
Multi-hop Payments: idea



- If Alice has a channel to Bob, and Bob has a channel to Carol, it is possible to make a payment between Alice and Carol by making two channel payments
- The challenge is how these transfers make them **atomic**

Multi-hop Payments: protocol

Consider the transfer of 1 BTC from A to C, with a fee of 1% to B



The Lightning Network



- Bitcoin's Lightning network implements a large network of open bi-directional payment channels
- The software provides ways to find payment routes in the network
 - Most payments happen in a few hops (1 or 2 intermediaries)
- It's a big network, where nodes make money from transfer fees

Taken on May 7th, 2026,
from <https://1ml.com/statistics>

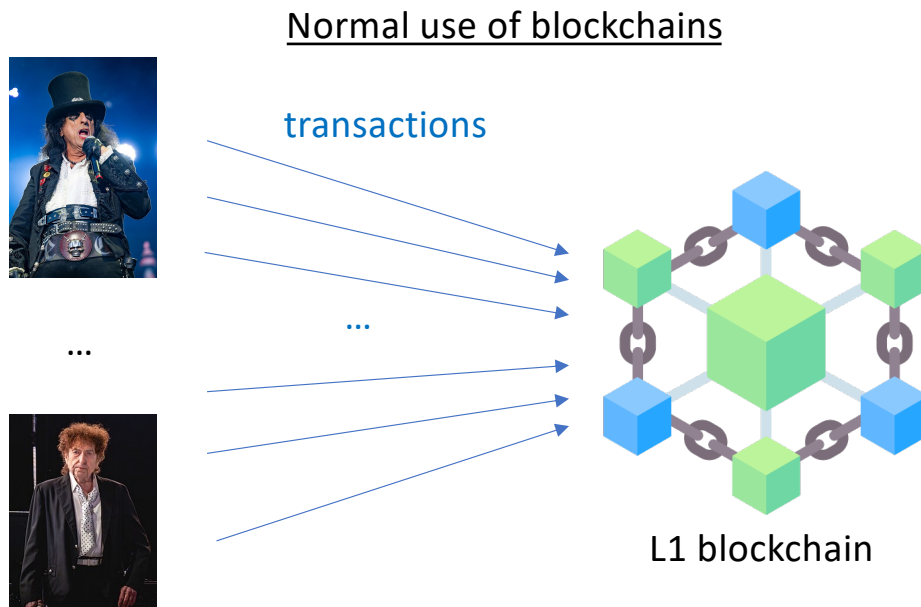
Number of Nodes	Number of Channels	Network Capacity
6,055 ↑+3.47%	19,765 ↑+6.0%	2,852.78 BTC ↑+3% \$227,928,323.58
Nodes Observed	New Nodes (24h)	New Channels (24h)
62,897 ↑+0.53%	5 ↓-28.57%	194 ↑+14.79%

DCB

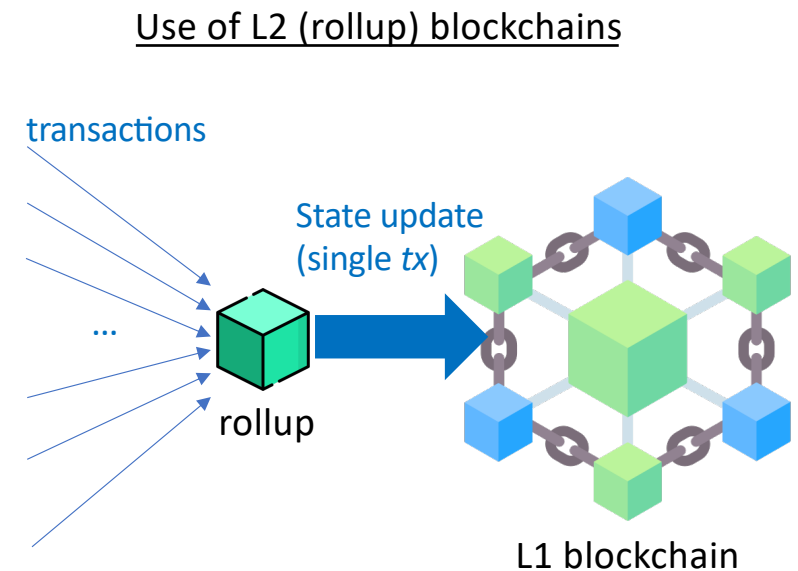
Rollups

General Idea

- Another way to process more transactions per second (with smaller fees) is to move the work elsewhere: rollups!



DCB



21

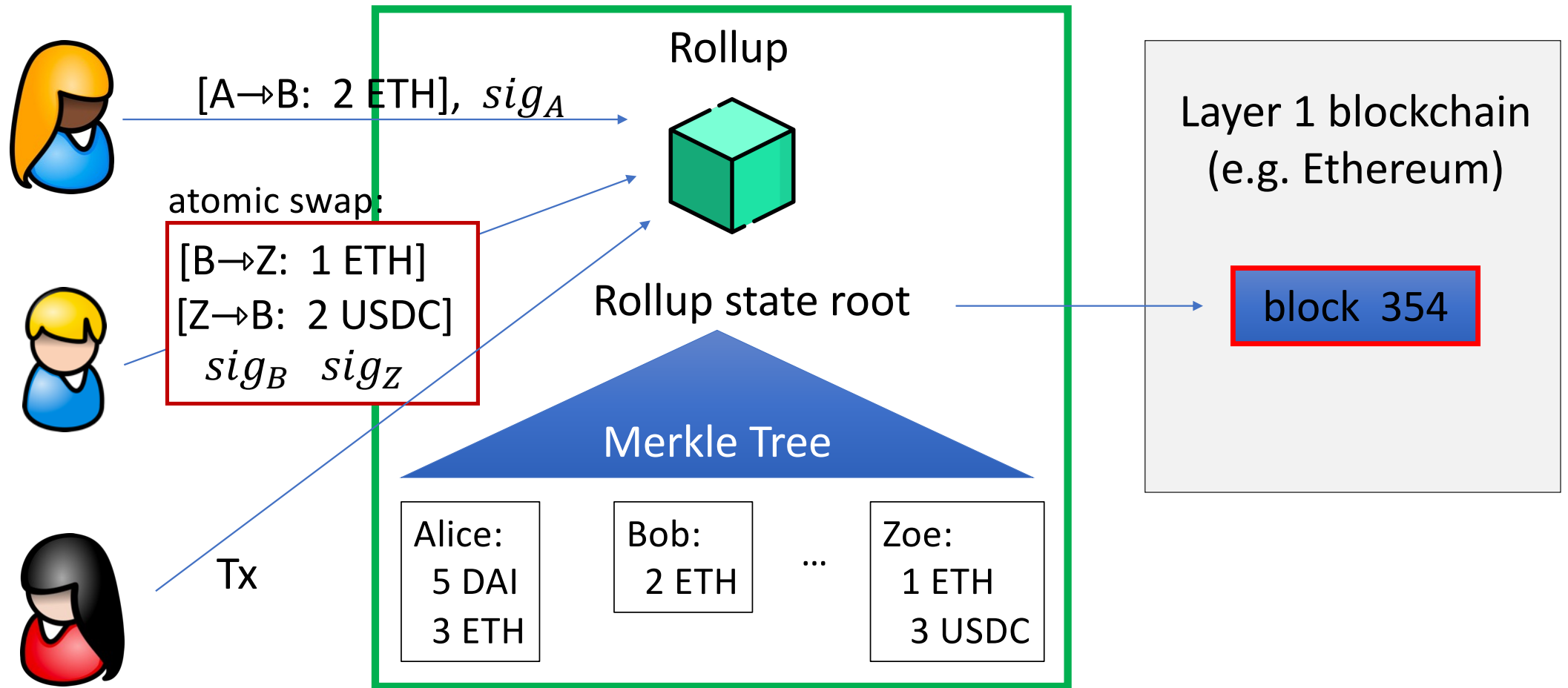
General Idea: complications

- Core Idea: execute a bunch of transactions on the rollup (e.g., a series of updates on Alice and Bob accounts, and, after some time, the rollup update their accounts on L1 with a single transaction
- Complications:
 - 1) How to make this transaction consume a small amount of gas?
 - 2) How to ensure the rollup do the update?
 - 3) How to ensure the rollup do the correct update?

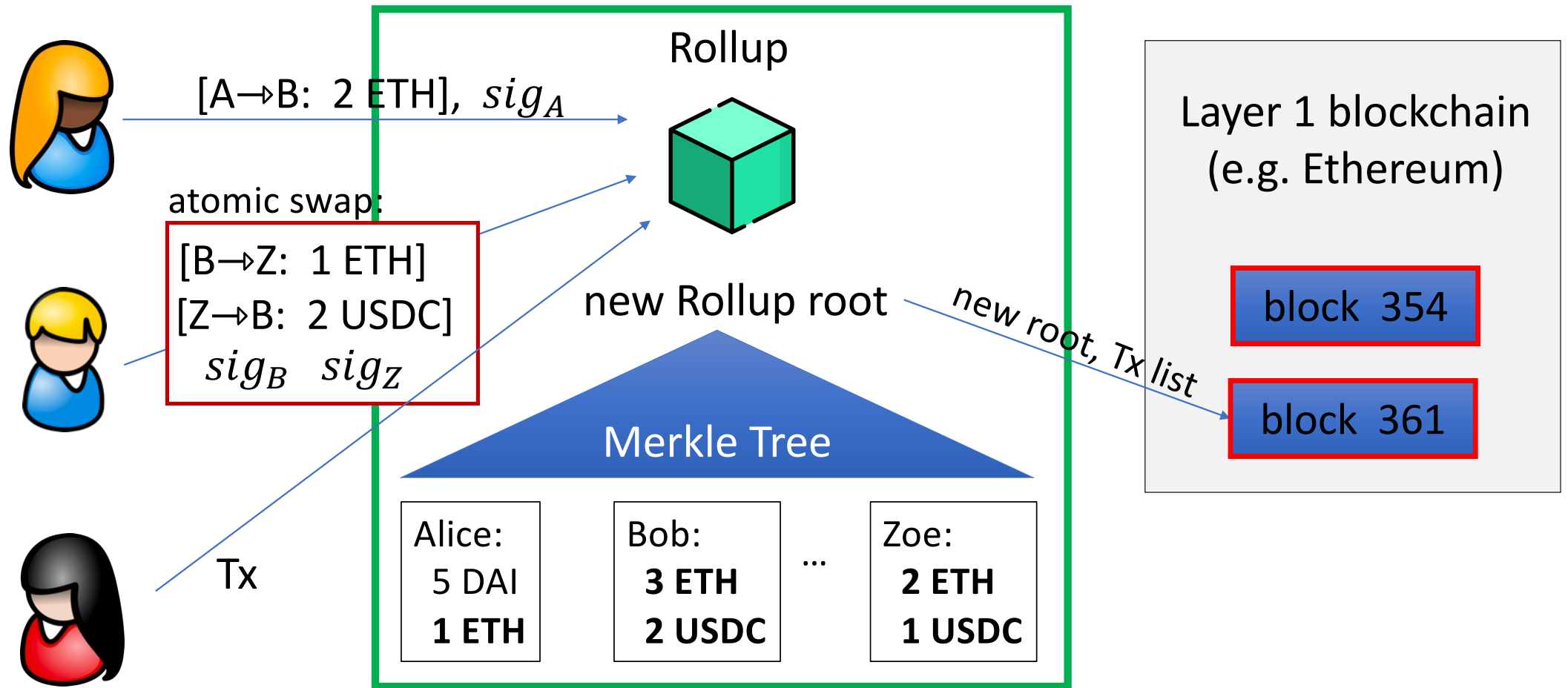
How to make the update transaction consume a small amount of gas?

- Solution 1: Just update the accounts that were touched by the transactions
 - E.g., if Alice, Bob and Carol did many transfers between themselves, just update their final balances at the end
 - This is somewhat like payment channels
 - This does not work for general smart contracts on L2 and many users, as this would still require a lot of state to be updated
- Solution 2: store only the Merkle tree root of the current rollup state together with the compressed executed transactions on L1 blockchain
 - This is the standard way rollups work

Rollup Operation

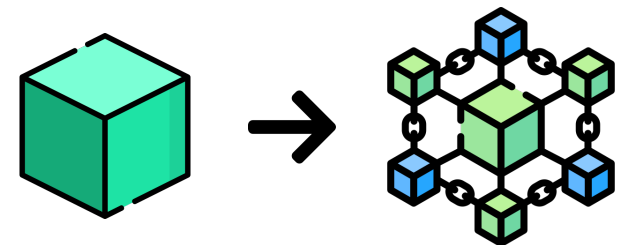


Rollup Operation



How to ensure the rollup do the update?

- What if the rollup is unavailable or just don't want to do the update?
- The Rollup is composed of at least three elements:
 - 1) The L1 blockchain contract
 - 2) The sequencer, for ordering rollup transactions
 - 3) The off-chain virtual machine for executing transactions and update L1
- (1) is mostly solved, but (2) and (3) requires replication
 - (2) is implemented by a total order broadcast among rollup nodes
 - (3) requires a rollup coordinator to update L1
 - Other nodes verify if the updates are correct
 - ... and replace the coordinator if they aren't



How to ensure the rollup do the correct update?

- Correct (state) update: an update that reflects the execution of the batch of valid transactions on the last submitted correct update state
- No-solution: L1 smart contract executes all transactions to verify if the state matches the provided Merkle tree root
- Solutions: Two solutions:
 - **Optimistic rollups**: coordinator updates L1 blockchain and everyone (including other rollup nodes) has a certain time (e.g., 2 days) to challenge the updated state with a complain (used in [Arbitrum](#) and [Optimism](#))
 - Drawback: takes time to revert invalid states
 - **ZK-rollups (Zero Knowledge)**: coordinator uses cryptographic proofs (SNARK) that are small and easy to verify to prove the update is correct; these proofs are verified by the L1 smart contract (used in [ZKsync](#) and [Starknet](#))
 - Drawback: generating such proofs is very expensive

(some) Bibliography

- Dan Boneh's slides (see cover)
- J. Poon & T. Dryja. The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments. <https://lightning.network/lightning-network-paper.pdf>
- Interdax. Scaling Ethereum on L2: Optimistic Rollups and ZK-Rollups. <https://medium.com/interdax/ethereum-l2-optimistic-and-zk-rollups-dffa58870c93>
- F. Mangone. Blockchain 101: Rollups. <https://medium.com/@francomangone18/blockchain-101-rollups-8da114a51f32>